

Vertex Event System Specification

Version: 1.0

Status: Draft

1. Objetivo

O Event System é o subsistema do Vertex responsável pela distribuição de notificações sobre acontecimentos ocorridos na plataforma.

Um evento representa algo que aconteceu, e não um comando, uma requisição ou um estado.

Seu objetivo é desacoplar produtores e consumidores de eventos, permitindo que múltiplos plugins e serviços reajam a mudanças sem conhecer diretamente uns aos outros.

«Importante: O Vertex não utiliza um Event Bus global tradicional. O Event System é um serviço do Core, controlado, tipado e com escopo definido, evitando os problemas comuns de Event Bus genéricos.»

2. Responsabilidades

O Event System é responsável por:

- registrar assinaturas de eventos;
- remover assinaturas;
- publicar eventos;
- entregar eventos aos inscritos;
- validar tipos de eventos;
- controlar prioridades;
- aplicar filtros;
- garantir isolamento entre produtores e consumidores;
- registrar métricas.

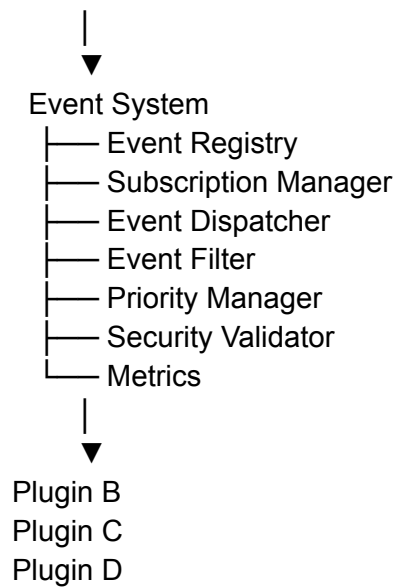
O Event System não mantém estado permanente.

3. Arquitetura

Plugin A



Event API



Os produtores não conhecem os consumidores.

4. Componentes

Event Registry

Mantém os tipos de eventos registrados.

Cada evento possui:

- identificador;
- versão;
- categoria;
- metadados.

Subscription Manager

Gerencia inscrições.

Responsável por:

- registrar;
- cancelar;
- consultar inscrições.

Event Dispatcher

Entrega eventos aos consumidores.

Suporta entrega:

- síncrona;
- assíncrona.

Event Filter

Permite filtrar eventos por:

- tipo;
- categoria;
- origem;
- prioridade.

Priority Manager

Ordena a entrega conforme a prioridade definida.

Security Validator

Verifica permissões antes da publicação ou consumo de determinados eventos.

Metrics Collector

Coleta estatísticas de utilização.

5. Fluxos

Publicação

Plugin

↓

publish()

↓

Dispatcher

↓

Consumidores

Assinatura

Plugin

↓

subscribe()

↓

Subscription Manager

↓

Inscrição ativa

Entrega

Evento

↓

Filtro

↓

Prioridade

↓

Consumidores

6. APIs

Publicação

- publish()

Assinatura

- subscribe()
- unsubscribe()

Consulta

- registeredEvents()
- subscriptions()

Controle

- pauseSubscription()
- resumeSubscription()

7. Estados

Evento

Created

↓

Published

↓

Dispatching

↓

Delivered

↓

Completed

Em caso de erro:

Failed

Assinatura

Active

↓

Paused

↓

Removed

8. Políticas

Evento representa passado

Eventos representam acontecimentos já ocorridos.

Nunca devem ser usados como comandos.

Sem persistência

Eventos são transitórios.

Após entregues, deixam de existir.

Caso informações precisem permanecer disponíveis, deve-se utilizar o State Engine.

Ordem

Eventos de mesma prioridade preservam a ordem de publicação.

Prioridades

Prioridades suportadas:

- Critical
- High
- Normal
- Low

Entrega

Cada tipo de evento define sua política:

- síncrona;
- assíncrona.

A maioria dos eventos utiliza entrega assíncrona.

Isolamento

Falhas em consumidores não impedem a entrega aos demais.

Segurança

Eventos sensíveis exigem permissões específicas.

9. Integrações

O Event System integra-se com:

- Plugin Manager;
- Plugin API;
- Plugin Context;
- Capability System;
- State Engine;
- Scheduler;
- Security Framework;
- Policy Engine;
- Resource Manager;
- Logging Framework;

- Configuration System;
- Plugin Inspector;
- System Event Core.

O Event System não substitui o State Engine nem o Capability System.

10. Métricas

São registradas:

- eventos publicados;
- eventos entregues;
- eventos descartados;
- assinaturas ativas;
- tempo médio de entrega;
- eventos por segundo;
- falhas de entrega;
- filtros aplicados;
- eventos mais utilizados;
- consumidores mais ativos.

11. Exemplos

Evento de bateria

battery.level.changed

Consumidores:

- Watchface;
- Economia de energia;
- Estatísticas.

Evento de mídia

media.started

Consumidores:

- Controles rápidos;
- Histórico;
- Interface.

Evento de plugin

plugin.installed

Consumidores:

- Plugin Store;
- Plugin Inspector.

Evento de sistema

system.shutdown

Consumidores:

- Scheduler;
- Storage;
- Logging.

12. Regras Arquiteturais

- Eventos representam fatos, nunca comandos.
- Eventos não armazenam estado.
- Toda publicação ocorre através da Event API.
- Plugins nunca enviam eventos diretamente para outros plugins.
- O Event Dispatcher é o único responsável pela distribuição.
- O sistema deve minimizar bloqueios, priorizando processamento assíncrono.

13. Limitações

O Event System não é responsável por:

- executar funcionalidades (Capability System);
- compartilhar estados (State Engine);
- controlar ciclo de vida;
- armazenar informações;
- gerenciar permissões de plugins.

14. Futuras Extensões

O Event System poderá evoluir para suportar:

- filtros por expressões;
- assinatura por padrões (wildcards);
- prioridade dinâmica;
- entrega agrupada (batch dispatch);
- rastreamento distribuído de eventos;
- eventos entre dispositivos Vertex.

Essas extensões devem preservar a simplicidade e o baixo custo do sistema.

15. Resumo

O Event System fornece um mecanismo leve, tipado e desacoplado para notificação de acontecimentos dentro da plataforma Vertex. Ao distribuir eventos sem criar dependências diretas entre produtores e consumidores, ele complementa o Capability System e o State Engine, formando um modelo de comunicação modular, seguro e eficiente, sem recorrer a um Event Bus global tradicional.